

Summary of Alpa and PartIR

Sami Fayyad (samiuf), David McDermott (dmcde), Yana Patel (yrpatel)

Problem and Motivation

Alpa

Recent advances in deep learning have been the result of intense scaling of model size, with models like GPT-3 encompassing billions of parameters at the time and being trained on massive input datasets. Conducting training with such large models and inputs requires specializing training plans to the specific distributed environments on which the models are being trained.

Achieving the correct tuning is of significant importance and can make orders of magnitudes of difference in final model performance. However, doing so takes significant expertise. Current model parallelization techniques require developers to either manually devise their own parallel execution plan or use a tool that selects from a limited set of options, failing to take full advantage of distributed compute meshes on which models are often trained. This necessitates automating parallelization to enable model designers to focus on optimizing the models themselves, instead of tinkering with training configurations.

PartIR

Though parallelization has become necessary to efficiently train neural network models that are scaling by ten times per year, existing methods to generate parallelization plans and shard operations across devices require significant manual configuration by model developers. This typically involves annotating model code with parallelization methods by hand, which is not only time-consuming but also dependent on the exact framework and environment in which a model is being created. Maintainability and portability suffer as a result, since there is no easy or automatic way to translate annotations as models get reconfigured or hardware specifications change. Manual configurations are also more prone to human error, which can go undetected and affect model performance.

Related Works

While Alpa and PartIR both attempt to combine all three of automation, inter-operator parallelism, and intra-operator parallelism, previous works touched upon a single one of those three aspects:

Megatron-LM (2019): One of the first implementations of tensor parallelism (splitting large operations across many GPUs). Demonstrated that tensor parallelism allowed scaling to billions

of parameters despite limited GPU memory, but required manual sharding and heavy communication between GPUs.

Mesh-TensorFlow (2018) and **G-shard** (2020): These papers provided higher level APIs to partition tensors across devices, somewhat simplifying the process, but required lots of detailed annotations throughout the model.

Gpipe (2019): First demonstration of inter-operator parallelism. Gpipe split layers across devices and fed data through the pipeline in microbatches, reducing memory footprint, but creating memory bubbles that led to idle GPUs and increased latency.

ZeRO (2020): Demonstrated automatic sharding of states, gradients, and parameters across devices, providing memory savings, but didn't provide pipeline parallelism strategies.

Other previous works provided two of the three:

Megatron-LM v2 (2021): A follow-up to the original 2019 paper, this paper provided a breakthrough in combining tensor and pipeline parallelism. However, the system required lots of manual configuration, which the authors said took them weeks.

PipeDream (2019) and **DAPPLE** (2020): These papers combined pipeline parallelism with automation to reduce the bubbles that had been present in Gpipe, but faced tradeoffs between weight staleness and throughput.

Tofu (2019) and **FlexFlow** (2018): These papers both performed automated searches of intra-operator combinations to find ideal model configurations. While effective, this search-based model is impractical for today's massive models.

Solution Overview

Alpa

Alpa automates the parallelization and compilation of a distributed learning from its IR graph. It does so by categorizing parallelization techniques into two categories: intra-operator parallelism (splitting operations that act along one or more tensor axes) and inter-operator parallelism (slicing the model into stages of a training pipeline). It takes advantage of the fact that the properties of these two types of parallelism fit into a hierarchical space that matches the layout of device meshes. Intra-operator parallelism has less idle time but has high communication overhead, as it requires synchronization between all partitioned devices at the end of each stage. Inter-operator parallelism, on the other hand, requires just peer-to-peer communication but introduces device idle time because of pipeline bubbles. Intra-operator steps can thus be mapped to nearby devices in the mesh, taking advantage of their efficient communication, while inter-operator steps can be assigned to devices with worse bandwidth.

To generate its parallelization plan from the input of an IR of graph stages and a device mesh configuration, Alpa first completes an **inter-operator pass**, where it iterates through combinations of device assignments to graph stages, enumerating the ways the input graph could be partitioned into stages, with each stage having various submesh configurations that it could be mapped to. It then sends these submesh configurations to an **intra-operator pass**, which uses an ILP algorithm to find the optimal configuration. The ILP algorithm minimizes the sum of the resharding cost and the communication/computation cost. Finally, given the ILP solution, it returns to the inter-operator algorithm to find the optimal assignment between stages and device meshes. It does so via a dynamic programming algorithm that optimizes for the sum of the latency of all stages and the latency of the longest stage multiplied by the number of batches.

The paper demonstrates that the performance of the parallelization plans Alpa creates match that of Megatron-LM (a manually optimized approach) for GPT-3, vastly outperform the state of the art DeepSpeed for Mixture of Experts Models, and provide the first scalable implementation of Wide-ResNet models.

PartIR

PartIR introduces a hardware-agnostic neural network partitioning system capable of producing both manual and automatic sharding strategies. Its approach gradually rewrites the IR of a training graph in a series of transparent steps, which is intended to be more predictable, accessible, and understandable than existing methods. As opposed to requiring detailed user annotations, the user specifies only the type of optimizations that they desire, and the compiler does a pass to craft the parallelization and sharding strategy.

PartIR's most interesting contribution is a **propagation pass** that determines which sets of instructions can be combined into single operations, freeing up GPUs to perform more parallel computations. It does so by identifying equivalences between loop and slice instructions and rewriting the intermediate steps to be performed as individual actions.

The paper evaluated the tool on three variations of a transformer model and compared it to the state-of-the-art GPSMD, finding similar performance despite the comparative ease with which its parallelization technique could be specified. It also compared four different models with various combinations of automatic and manual parallelizations, and found the runtime of a purely automatic strategy to be better than or comparable to the runtimes of the manual configurations in all four cases. Finally, it benchmarked partition time as a portion of overall compilation time and found it to be insignificant for both fully automatic and hybrid-manual-automatic models.

The key contribution is separating the partitioning configuration from the model development through a simple API, allowing model developers to focus on improving the model itself, instead of partitioning its training.

Limitations

Alpa

Alpa omits certain costs of computation from its model to avoid an exponential blowup of states and passes. One such cost is communication between pipeline stages because it is usually negligible in the context of the entire computation. This communication can also be overlapped with performing computation for the next pipeline stage on the sending device, keeping the compute device saturated. Indeed, followup discussion and clarification recognized that it is not problematic to omit this from the cost model because of the communication typically taking place within a local GPU cluster, meaning that available network bandwidth leaves a lot of room.

Alpa is not a completely automatic tool. It demands a hyperparameter (B , the number of micro-batches) as input, but the authors note that architects can leverage the tool with different values of B inputted themselves. However, this makes a case for the tool not providing total automation, which limits the goals that it can achieve but enhances potential future work.

Unfortunately, Alpa's pipeline parallelism setup only considers static linear schedules, without considering the opportunity to squeeze out extra parallelism as in pipeline schedules like 1F1B or more impressively parallel schedules.

Currently Alpa's only capable of dealing with static graphs, which means it cannot optimize for having a device overlap communication and computation, sacrificing potentially crucial concurrency gains. The more that a user knows at compile time, the better they can leverage Alpa for optimization.

PartIR

PartIR cannot handle reshapes well because of an IR blow-up problem. This IR blow-up scales with the number of devices according to the authors, demonstrating that PartIR has scalability limitations. This note is particularly relevant to the in-class discussion regarding scaling these tools to real-world companies with massive GPU clusters. It seems that extending some aspects of PartIR's implementation might be limited by fundamental issues when it comes to handling reshaping. Additionally interesting is that propagation gets blocked when devices do not perfectly divide the partitioned dimension on which the devices are operating, leaving some optimizations on the table for simplicity.

PartIR struggles with explicit value replication and unfortunately needs to accomplish this by making a loop that blocks computation instead of replicating values with a more elegant solution. This "worse is better" solution does the job, but it leaves compute on the table and commits a costly compute sin by keeping a value around by stalling computation instead of finding a genuine solution to replicate the value.

Future Research Directions

Alpa

Alpa's cross-mesh sharding requires a communication plan that takes place in two iterations, with a peer-to-peer transfer across meshes followed by an all-gather on any devices in the second mesh that require a tensor be shared. The authors acknowledge that this system is a heuristic, and future work could investigate the optimal method for cross-mesh resharding.

PartIR

Future research could extend the PartIR model to support training over heterogeneous clusters using Multiple Program Multiple Data (MPMD) systems, which have different machines running entirely different programs on different input data sets. This model requires different partitioning strategies, which the existing PartIR framework doesn't address.

Summary of Class Discussion

Alpa

Q: Is compilation time a potential con?

A: Takes ~40 minutes for large models, and compared to the time spent training that's insignificant.

Q: ILP is NP-hard, and the size of these graphs is getting larger. The testing was done on a 64 GPU cluster, which is much smaller than systems at companies like Google and Meta. Would ILP get too complex? The paper says that compilation time scales linearly with GPU, but their DP algorithm and ILP itself do *not* scale linearly.

A: ILP is based on the number of operators on the graph, not the number of GPUs. As long as the computational graph stays a similar size, which is what has been happening, don't expect the ILP to become inefficient. The paper claims the ILP is efficient to graphs with tens of thousands of stages of operators.

Q: Does Alpa take care of batches for the user, or must the user specify?

A: Provides some constraints on the minimum number of microbatches you can have because it has a certain number of stages in the pipeline.

Q: Is Alpa nondeterministic? Don't ILP solvers have some randomness?

A: It should be consistent if run twice with the same environment, model, and parameters. In some corner cases – depending on what solver you're using – you may get random behavior, but in practice it should be consistent.

Q: One limitation the authors mention is that they don't model the cross-stage communication cost because they expect it to be small. On a larger scale, with more than 64 GPUs, could this assumption no longer hold?

A: It's a limitation, but would not expect significant slowdowns. Between stages, you typically have peer-to-peer (P2P) communication, not collective communication. Given that network bandwidth is increasing, and given that P2P has linear complexity, it shouldn't be a problem. If you had extreme network constraints, perhaps, but many AI servers have as many NICs as GPUs, so each GPU has lots of bandwidth.

Q: They say their execution plans are "near" optimal. What would it take to be truly optimal, and how far off are they?

A: The limitation is that the DP algorithm takes some shortcuts because of how many different possible combinations of stages and device assignments there are. Can't iterate through every combinational graph, so they only run through a subset of what's possible. It's an NP-hard problem, so they also do not know and cannot demonstrate it is optimal. We don't know how far from optimal it is.

Q: Multimodal AI often has different computational graphs. How could these frameworks be applied to multimodal AI?

A: Alpa is generalizable and model agnostic, so as long as you can create a computational graph from the model, Alpa should be able to deal with it and optimize for it. Alpa could run into issues with dynamic graphs, but it should work on static ones. There would be room for improvements.

PartIR

Q: How are operations defined? Who decides what operations are done?

A: This happens at the compiler pass. An entry represents a line of code, and the dimensions and operation of a line of code are both known. A line tells us a type of operation that will be performed to divide data across GPUs. The compiler validates two types of operations: the shape of things after a slice occurs, and the type of operation actually being performed to get the data on the device.

Q: Does it handle pipeline parallelism as well?

A: It does, though the paper doesn't go into detail.

Q: The authors were doing some sort of automation in partition, making it a hybrid of manual and automatic. They used Monte Carlo tree search in the automatic partitioning, which could take up to 20 minutes, which is around the same amount of time as manual. As models get larger and larger, could this become too slow to optimize?

A: Yes. The paper provides an ability for those who do not know how to manually optimize their models to have a means to do so. Their goal is matching performance while providing ease of access, not outperforming manual partitioning. In addition, as models get larger, though it's true that PartIR's process may take longer, it will become impossible for humans to partition manually.

Q: Who are these frameworks targeted for?

A: Engineers who don't want to spend as much time developing parallelization strategies. It takes several hours or more to do manual configurations. PartIR aims to minimize the amount of time spent on that, so developers can focus their efforts on actually developing models.

Q: Jax is a static computational graph that can be traced and optimized, but other libraries like PyTorch are dynamic graphs. Wouldn't the logic not even apply?

A: It's an implementation problem, not an idea problem. Jax is the toolchain where this is easy and feasible. Maybe it's not possible on PyTorch, but it's probably doable if someone were to put in a lot of time; unfortunately, there's no incentive to do so.